Market-Basket Analysis

May 11, 2026

[1]:
```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

[2]:
```
df = pd.read_csv("Groceries_dataset.csv")
df.head()
```

[2]:

	Member_number	Date	itemDescription
0	1808	21-07-2015	tropical fruit
1	2552	05-01-2015	whole milk
2	2300	19-09-2015	pip fruit
3	1187	12-12-2015	other vegetables
4	3037	01-02-2015	whole milk

[3]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 38765 entries, 0 to 38764
Data columns (total 3 columns):
 #   Column           Non-Null Count  Dtype 
---  ------           --------------  ----- 
 0   Member_number    38765 non-null  int64 
 1   Date             38765 non-null  object
 2   itemDescription  38765 non-null  object
dtypes: int64(1), object(2)
memory usage: 908.7+ KB
```

[4]:
```
df.isnull().sum().sort_values(ascending = False)
```

[4]:
```
Member_number      0
Date               0
itemDescription    0
dtype: int64
```

[5]:
```
df["date"] = pd.to_datetime(df["Date"])
df.drop("Date", axis = 1, inplace = True)
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 38765 entries, 0 to 38764
Data columns (total 3 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Member_number    38765 non-null  int64
 1   itemDescription  38765 non-null  object
 2   date             38765 non-null  datetime64[ns]
dtypes: datetime64[ns](1), int64(1), object(1)
memory usage: 908.7+ KB
```

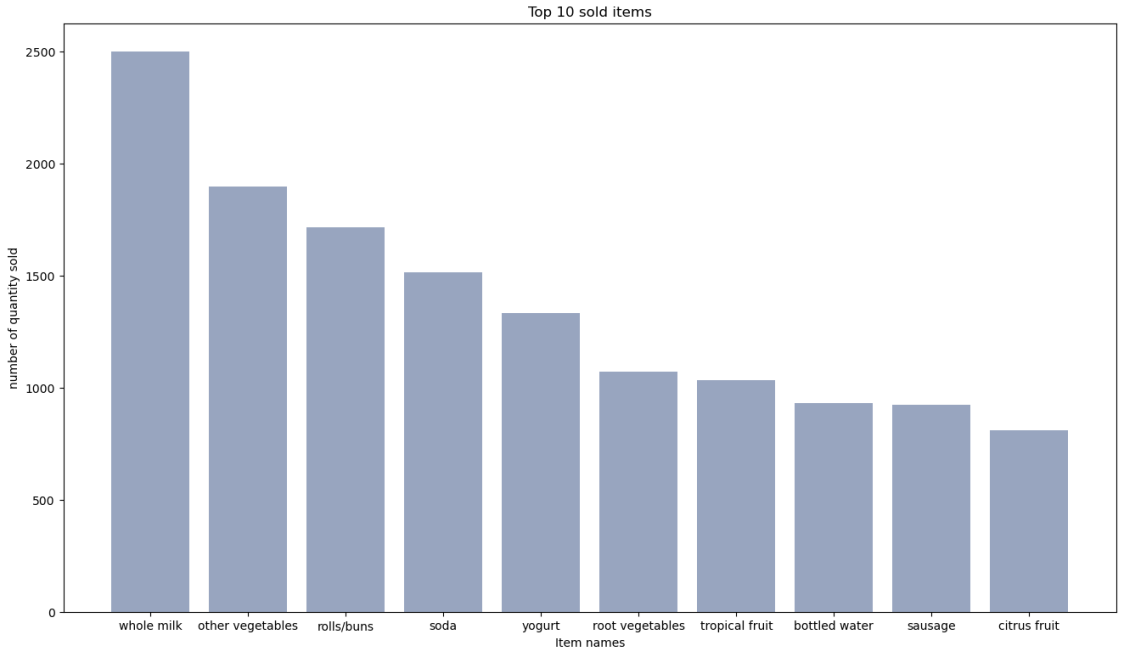
```
C:\Users\HOME\AppData\Local\Temp\ipykernel_7600\3019347331.py:1: UserWarning:
Parsing dates in DD/MM/YYYY format when dayfirst=False (the default) was
specified. This may lead to inconsistently parsed dates! Specify a format to
ensure consistent parsing.
  df["date"] = pd.to_datetime(df["Date"])
```

[6]:
```
df.head()
```

[6]:

	Member_number	itemDescription	date
0	1808	tropical fruit	2015-07-21
1	2552	whole milk	2015-05-01
2	2300	pip fruit	2015-09-19
3	1187	other vegetables	2015-12-12
4	3037	whole milk	2015-01-02

[7]:
```
Item_dist = df.groupby(by = "itemDescription").size().reset_index(name =␣
 ↪"frequency").sort_values(by = "frequency", ascending = False).head(10)
bars = Item_dist["itemDescription"]
height = Item_dist["frequency"]
x_pos = np.arange(len(bars))
plt.figure(figsize = (16, 9))
plt.bar(x_pos, height, color = (0.2, 0.3, 0.5, 0.5))
plt.title("Top 10 sold items")
plt.xlabel("Item names")
plt.ylabel("number of quantity sold")
plt.xticks(x_pos, bars)
plt.show()
```



[8]:
```
df_date = df.set_index(["date"])
df_date
```

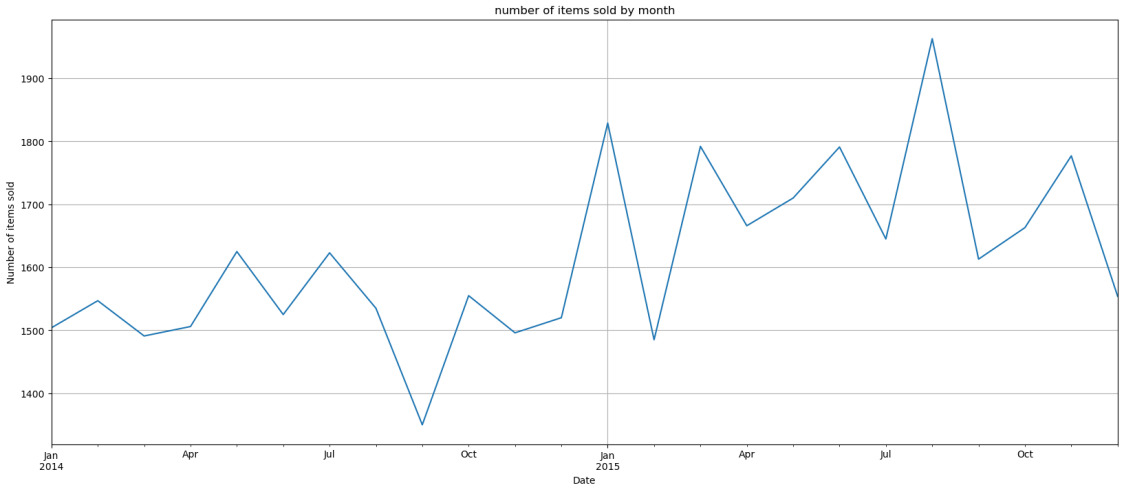
[8]:

date	Member_number	itemDescription
2015-07-21	1808	tropical fruit
2015-05-01	2552	whole milk
2015-09-19	2300	pip fruit
2015-12-12	1187	other vegetables
2015-01-02	3037	whole milk
…	…	…
2014-08-10	4471	sliced cheese
2014-02-23	2022	candy
2014-04-16	1097	cake bar
2014-03-12	1510	fruit/vegetable juice
2014-12-26	1521	cat food

[38765 rows x 2 columns]

[9]:
```
df_date.resample("M")["itemDescription"].count().plot(figsize = (20, 8), grid =␣
↪True, title = "number of items sold by month").set(xlabel = "Date", ylabel =␣
↪"Number of items sold")
```

[9]: [Text(0.5, 0, 'Date'), Text(0, 0.5, 'Number of items sold')]



[10]:
```
cust_level = df[["Member_number", "itemDescription"]].sort_values(by =␣
 ↪"Member_number", ascending = False)
cust_level["itemDescription"] = cust_level["itemDescription"].str.strip()
cust_level
```

[10]:

	Member_number	itemDescription
3578	5000	soda
34885	5000	semi-finished bread
11728	5000	fruit/vegetable juice
9340	5000	bottled beer
19727	5000	root vegetables
…	…	…
13331	1000	whole milk
17778	1000	pickled vegetables
6388	1000	sausage
20992	1000	semi-finished bread
8395	1000	whole milk

[38765 rows x 2 columns]

[11]:
```
transactions = [a[1]["itemDescription"].to_list() for a in list(cust_level.
 ↪groupby("Member_number"))]
```

[12]:
```
from apyori import apriori
rules = apriori(transactions = transactions, min_support = 0.002,␣
 ↪min_confidence = 0.05, min_lift = 3, min_length = 2, max_length = 2)
```

[13]:
```
results = list(rules)
```

[14]:
```
results
```

[14]:
```
[RelationRecord(items=frozenset({'kitchen towels', 'UHT-milk'}),
support=0.002308876346844536,
ordered_statistics=[OrderedStatistic(items_base=frozenset({'kitchen towels'}),
items_add=frozenset({'UHT-milk'}), confidence=0.30000000000000004,
lift=3.821568627450981)]),
 RelationRecord(items=frozenset({'potato products', 'beef'}),
support=0.002565418163160595,
ordered_statistics=[OrderedStatistic(items_base=frozenset({'potato products'}),
items_add=frozenset({'beef'}), confidence=0.4545454545454546,
lift=3.8021849395239955)]),
 RelationRecord(items=frozenset({'canned fruit', 'coffee'}),
support=0.002308876346844536,
ordered_statistics=[OrderedStatistic(items_base=frozenset({'canned fruit'}),
items_add=frozenset({'coffee'}), confidence=0.4285714285714286,
lift=3.7289540816326534)]),
 RelationRecord(items=frozenset({'meat spreads', 'domestic eggs'}),
support=0.0035915854284248334,
ordered_statistics=[OrderedStatistic(items_base=frozenset({'meat spreads'}),
items_add=frozenset({'domestic eggs'}), confidence=0.4,
lift=3.0042389210019267)]),
 RelationRecord(items=frozenset({'mayonnaise', 'flour'}),
support=0.002308876346844536,
ordered_statistics=[OrderedStatistic(items_base=frozenset({'flour'}),
items_add=frozenset({'mayonnaise'}), confidence=0.06338028169014086,
lift=3.3385991625428253), OrderedStatistic(items_base=frozenset({'mayonnaise'}),
items_add=frozenset({'flour'}), confidence=0.12162162162162163,
lift=3.338599162542825)]),
 RelationRecord(items=frozenset({'rice', 'napkins'}),
support=0.0030785017957927143,
ordered_statistics=[OrderedStatistic(items_base=frozenset({'rice'}),
items_add=frozenset({'napkins'}), confidence=0.2448979591836735,
lift=3.011395094315329)]),
 RelationRecord(items=frozenset({'waffles', 'sparkling wine'}),
support=0.002565418163160595,
ordered_statistics=[OrderedStatistic(items_base=frozenset({'sparkling wine'}),
items_add=frozenset({'waffles'}), confidence=0.21739130434782608,
lift=3.1501535477614353)])]
```

[15]:
```
def inspect(results):
    lhs = [tuple(result[2][0][0])[0] for result in results]
    rhs = [tuple(result[2][0][1])[0] for result in results]
    supports = [result[1] for result in results]
    confidence = [result[2][0][2] for result in results]
    lifts = [result[2][0][3] for result in results]
    return list(zip(lhs, rhs, supports, confidence, lifts))
resultsInDataFrame = pd.DataFrame(inspect(results), columns = ["Left Hand␣
 ↪Side", "Right Hand Side", "Support", "Confidence", "Lift"])
```

[16]:
```
resultsInDataFrame.nlargest(n = 10, columns = "Lift")
```

[16]:

	Left Hand Side	Right Hand Side	Support	Confidence	Lift
0	kitchen towels	UHT-milk	0.002309	0.300000	3.821569
1	potato products	beef	0.002565	0.454545	3.802185
2	canned fruit	coffee	0.002309	0.428571	3.728954
4	flour	mayonnaise	0.002309	0.063380	3.338599
6	sparkling wine	waffles	0.002565	0.217391	3.150154
5	rice	napkins	0.003079	0.244898	3.011395
3	meat spreads	domestic eggs	0.003592	0.400000	3.004239

[ ]:
```

```